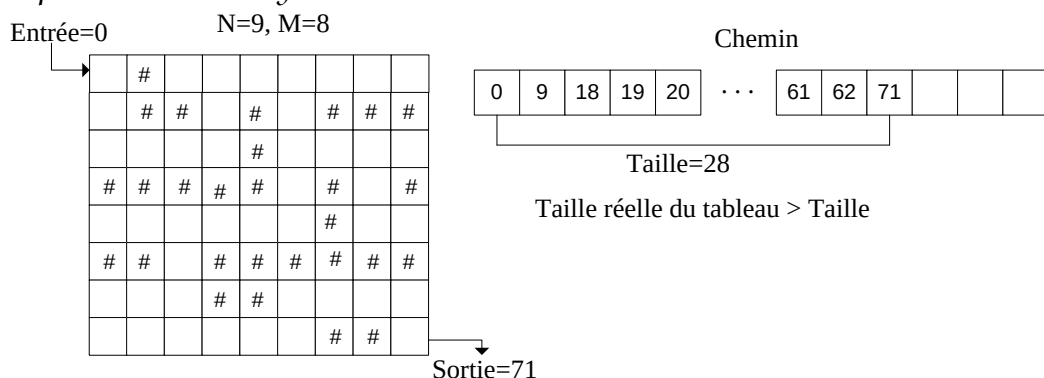


TD2 – Back-tracking

Exercice 1 : Labyrinthe

Le problème est représenté sous la forme d'une matrice $N \times M$ (de char ' ') dans laquelle les murs sont représentés par une valeur particulière (exemple le caractère #). On souhaite partir de l'entrée (qui se trouve dans la case 0) trouver un chemin qui nous mène vers la sortie. Quatre directions sont autorisées : haut, bas, droite et gauche.

- On utilisera pour résoudre le problème les variables N , M , **Labyrinthe** (un tableau de char $N \times M$ déjà rempli), **Chemin** (un tableau d'entiers, qui contiendra les cases parcourues) et la **Taille** de ce tableau. Cette Taille sera égale à 1 au début de l'algorithme (position 0) et sera incrémentée chaque fois que l'on avancera dans le labyrinthe et décrémentée chaque fois que l'on sera amené à faire marche arrière.
- Ecrire en pseudo code les algorithmes des fonctions **Existe(val)**, **AjouterFin(val)** et **SupprimerFin()** qui teste si une valeur **val** se trouve dans le tableau **Chemin**, ajouter la valeur en tant que dernier élément dans le tableau et supprimer le dernier élément du tableau ;
- Proposez un algorithme en pseudo code qui permet d'obtenir une solution au problème du labyrinthe.



- A partir du chemin calculé dans la question précédente remplacer dans le tableau **Labyrinthe** les blancs par un point et afficher le résultat sous la forme d'une matrice $N \times M$.

.	#		.	.	.			
.	#	#	.	#	.	#	#	#
.	.	.	.	#	.			
#	#	#	#	#	.	#		#
		.	.	.	.	#		
#	#	.	#	#	#	#	#	#
		.	#	#	.	.	.	.
		.	.	.	.	#	#	.

Exercice 2 : Labyrinthe

Modifier le programme précédent pour recherche le meilleur chemin entre les points de départ et d'arrivée. Le meilleur chemin est celui est le plus court.

Exercice 2 : Les interrupteurs

Un interrupteur est ouvert si certaines conditions sont remplies. Ces conditions portent sur des variables de type booléen, qui représentent les positions droite ou gauche. Des règles composées des opérateurs logiques (and, or et not) appliquées sur les variables indiquent si l'interrupteur est ouvert ou fermé.

Le principe de l'exercice consiste à savoir si pour un ensemble d'interrupteurs N liées à M variables, il existe une suite de position pour ces variables qui ouvrent tous les interrupteurs.

Exemple : N=5 (I1 à I5) et M=5 (V1 à V5)

I1 : V1 and not V5	ou	and V1 not V5
I2 : V2 or (V3 and V4)	ou	or V2 and V3 V4
I3 : not V1 or not V2	or	or not V1 not V2
I4 : (V2 and V4) or (V3 and not V5)	or	or and V2 V4 and V3 not V5
I5 : V3 or (V2 and V5)	or	or V3 and V2 V5

Dans ces circonstances (V1,V2,V3,V4,V5)=(1, 0, 1, 1, 0) est une solution pour ouvrir les 5 interrupteurs.

Implémentation.

- Proposez des structures permettant d'enregistrer les valeurs M variables, ainsi que les N expressions des différents interrupteurs. Les expressions sont enregistrées en notation polonaise.
- Grâce à la fonction `split` découper les expressions en N tableaux composés des différents opérateurs et de opérandes.
- Créer une fonction qui teste si une configuration de N expressions pour les M variables autorise l'ouverture de tous les interrupteurs.

- Implémentez une fonction qui recherche quel est la configuration permettant d'ouvrir tous les interrupteurs.
- Pour cela vous devez récursivement tester pour chaque variable (étape *e*) tester dans un premier temps si l'on doit arrêter la récursion. Celle-ci s'arrête si l'étape *e* a atteint le dernier interrupteur, ou si la configuration courante ouvre toutes les portes.
- Dans le cas contraire on doit identifier pour une étape donnée quels sont les valeurs possibles pour la variable *e*, modifier cette variable en conséquence et passer à l'étape suivante.

Exercice 3 : Chemin d'infiltration de l'eau

L'infiltration peut être définie comme l'écoulement d'un fluide dans un sol poreux. Un terrain n'est pas étanche s'il existe un chemin permettant au fluide de le traverser. Dans cet exercice, on souhaite proposer un modèle simplifié, qui test l'étanchéité d'un terrain. Un terrain sera représenté par une structure de type tableau à deux dimensions dont les cases (ou zones) peuvent être considérées comme non poreuse (représenté par exemple par le nombre 1) ou poreuse (valeur 0).

- Implémentez une fonction `creatGround(width, depth, percent)` qui crée un terrain (matrice `numpy`) de la taille (`width, depth`) contenant un pourcentage `percent` de zone non poreuses (valeur 1). Pour cela vous pouvez créer au début un terrain composé uniquement de zones poreuses.
- Vous pourrez ensuite récupérer la liste des cases poreuses, c'est-à-dire dans notre cas toutes les cases. Ensuite vous pourrez sélectionner dans cette liste les différentes cases à rendre non poreuse correspondant au pourcentage indiqué.
- La fonction `choice` de la bibliothèque `random` de `numpy` peut être utilisée à cet effet pour sélectionner en une instruction le nombre de valeur désirées. Il ne reste alors qu'à remplacer rendre poreuses les cases choisies.

On considère que l'eau ne peut s'écouler entre deux zones que si celles-ci sont voisines horizontalement ou verticalement. Il n'y aura donc pas d'écoulement en oblique. On suppose aussi que de l'eau peut arriver depuis le haut du terrain et l'apport est infini, c'est à dire que si une zone se remplit d'eau (valeur 2), il ne

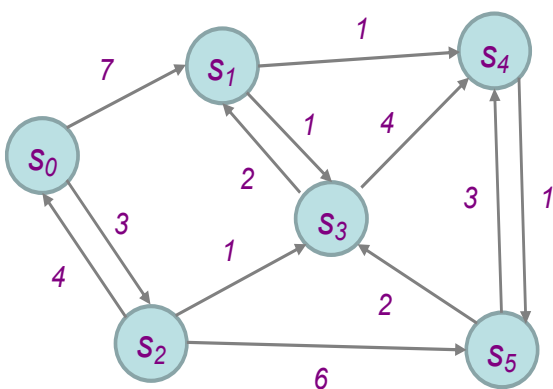
vide pas la zone qui l'a rempli. Ainsi, une zone qui est remplie d'eau ne se videra jamais. Les limites du terrain ne sont pas poreuses.

- Ecrire une fonction **accept** qui reçoit les coordonnées d'une case et retourne vrai si l'eau peut se propager dans cette case (valeur 0) et faux sinon.
- Implémentez maintenant une fonction récursive **infiltration(x,y)** qui reçoit une case du haut du terrain et remplit toutes les cases qui lui sont liées. A la fin de cette fonction toutes les cases remplies contiendront la valeur 2 (indiquant la présence d'eau).
- On souhaite connaître comment s'infiltre l'eau dans le terrain. Pour cela vous devez rechercher toutes les infiltrations à partir de toutes les entrées possibles en haut du terrain. Implémentez cette fonction.
- On souhaite trouver quel sont les différentes cavités de notre terrain, c'est-à-dire l'ensemble des zones adjacentes qui restent étanches. Proposez une fonction qui recherche toutes les cavités. Cette fonction retournera la liste des cavités. Une cavité est une liste de zone. Vous utilisez la valeur 3 pour indiquer que la zone a déjà été visitée.

Exercice 4 : Plus court chemin

Le programme consiste à rechercher dans un réseau routier un chemin entre une ville de départ D et une ville d'arrivée A. A la fin du programme on devra soit afficher qu'il n'existe aucune route entre D et A, et dans le cas contraire on affichera la liste des villes à parcourir entre D et A et le nombre de Km.

Exemple : Un réseau est représenté par une matrice carré $n*n$ sous la forme.



Réseau :

	S0	S1	S2	S3	S4	S5
S0	0	7	3	0	0	0
S1	0	0	0	1	1	0
S2	4	0	0	1	0	6
S3	0	2	0	0	4	0
S4	0	0	0	0	0	1
S5	0	0	0	2	3	0

$\text{Réseau}[0,1]=7$ indique qu'il existe une route entre la ville S_0 et la ville S_1 qui est de 7km. Par contre $\text{Réseau}[0,3]=0$ correspond au fait qu'il n'existe pas de route entre les villes S_0 et S_3 . On cherche le plus court chemin entre une ville de départ D et une ville d'arrivée A . Pour cela on se base sur l'algorithme de backtracking vu en cours (Page 10).

Description de l'algorithme.

- Parmi les deux algorithmes présentés quel est celui à utiliser.
- On suppose que l'on dispose des paramètres initiaux suivants : 1 – le réseau routier, 2 – la liste des villes [S_1 .. S_5], 3 – la ville de départ D et la ville d'arrivée A . Pour calculer le plus court chemin de quels types de paramètres avons-nous besoin.
- Dans le programme à quoi correspondra l'étape ei lors des appels de la fonction
- À quel moment on considère que la solution est trouvée, et dans ce cas que doit-on faire de cette solution.
- Quelles sont les étapes suivantes à partir de ei , et à quelles conditions une nouvelle étape est-elle acceptable.
- Si l'étape est acceptable comment doit-on modifier nos paramètres, avant d'appeler notre fonction récursivement.
- Que fait-on faire une fois que l'appel récursif se termine.

Implémentation.

- La première étape consiste à initialiser le réseau, pour cela vous utiliserez un fichier de texte contenant la liste des villes sur une première ligne et ensuite le tableau correspondant au réseau.
- Implémentez maintenant la fonction de recherche du plus court chemin. Vous devez obtenir à l'arrivée la liste des sommets parcourus ainsi que le nombre de km correspondant.

Exercice 5 : Le compte est bon

Le principe consiste à trouver grâce à quatre types d'opérations (+, -, *, /) un nombre compris entre 100 et 999. Seuls les entiers sont acceptés dans les opérations. Pour cela on dispose de 6 nombres choisis aléatoirement dans la liste des 24 nombres

suivants : [1,1,2,2,3,3,4,4,5,5,6,6,7,7,8,8,9,9,10,10, 25,50,75,100]. On doit alors pour résoudre ce problème disposer d'une liste (*Valeurs*) contenant les 6 nombres initiaux, de la liste des 4 opérations que l'on peut effectuer ainsi que du résultat à obtenir.

Le principe de l'algorithme récursif va consister à retirer deux nombres de la liste *Valeurs*, à effectuer une des 4 opérations sur ces deux nombres et à remettre le résultat dans la liste.

Description de l'algorithme.

- Parmi les deux algorithmes présentés quel est celui à utiliser.
- En plus de la liste *Valeurs* de quel type de variable avons-nous besoin si l'on veut in fine connaître la suite des opérations ayant permis de trouver le résultat recherché.
- A quelles conditions l'algorithme doit-il s'arrêter.
- Pour passer à l'étape suivante il est nécessaire de choisir deux nombres dans la liste et ensuite de vérifier quelles sont les opérations possibles entre ces deux nombres. Pensez-vous si vous on a déjà tester une opération entre deux nombres (ex : $N1+N2$) qu'il soit nécessaire de tester l'opération inverse ($N2+N1$). Qu'est-ce que lorsque l'on doit choisir quelles sont les opérations acceptables pour deux nombres.
- Si l'étape est acceptable comment doit-on modifier nos paramètres, avant d'appeler notre fonction récursivement.
- Que fait-on faire une fois que l'appel récursif se termine.

Implémentation.

- Choisissez aléatoirement une comprise entre 100 et 999 le Résultat à rechercher, ainsi que 6 valeurs aléatoire dans la liste des 24 nombres. Vous pourrez utiliser pour cela la fonction `choice` de `numpy` qui retourne `n` valeurs aléatoires dans une liste. Attention un élément ne peut être choisi qu'une fois (`replace=False`).
- Développez une fonction qui retourne les opérations que l'on accepte de faire à partir de deux valeurs ($N1$ et $N2$). Comme indiqué précédemment si l'on a déjà effectué une recherche avec une opération entre $N1$ et $N2$, il n'est pas nécessaire de faire une recherche dans le sens opposé. Afin d'éviter les deux opérations symétriques on peut choisir de ne faire une

opération que lorsque $N1 > N2$. Evitez également les opérations non pertinentes (division non entière ou résultat nul).

- Créez une fonction qui effectue l'opération lorsque l'on reçoit un opérateur ('+', '-', '*', ou '/') et deux valeurs.
- Lors du choix de deux variables pour l'opération future vous pourrez utiliser la fonction **permutations** de la bibliothèque **itertools**. Cette fonction renvoie les arrangements successifs de longueur r des éléments d'une liste. Exemple `permutations([a, b, c, a], 2)` -> ab, ac, aa, ba, bc, ba, ca, cb, ca, aa, ab, ac
- Implémentez la fonction du compte est bon. Vous conserverez la suite des opérations dans une liste dans une liste spécifique. Attention comme la liste des valeurs est modifiée après une opération (retrait de deux valeurs et ajout du résultat de leur opération) vous devez avant l'appel récursif dans la fonction créer une copie en profondeur (via **deepcopy** de la bibliothèque **copy**) de votre liste et effectuer les actions (suppressions des deux valeurs et ajout du résultat) sur cette liste. C'est cette nouvelle liste qui sera transmise lors de l'appel récursif.

Exercice 6 : Sac à dos

Étant donné une série d'objets possédant chacun un poids et une valeur, et un sac à dos de capacité limitée. La question est de savoir quels sont les objets que l'on peut mettre dans le sac à dos tout en maximisant la valeur transportée.

Exemple : Sac à dos (taille = 15kg)

Objets	1	2	3	4	5	6	7	8	9	10	11	12
Prix	4	4	3	3	5	5	3	3	5	6	5	5
Poids kg	5	7	2	2	6	8	3	4	9	11	8	7

Description de l'algorithme.

- Parmi les deux algorithmes présentés quel est celui à utiliser.
- De quels types de variables avons-nous besoins en plus de la liste des objets avec leurs prix et leurs poids, de la taille du sac.
- Dans le programme à quoi correspondra l'étape *ei* lors des appels de la fonction récursive.

- *A quelles conditions doit-on arrêter la récursivité.*
- *Quel sont les étapes suivantes à partir de ei, et à quelles conditions une nouvelle étape est-elle acceptable.*
- *Si l'étape est acceptable comment doit-on modifier nos paramètres, avant d'appeler notre fonction récursivement.*
- *Que fait-on faire une fois que l'appel récursif se termine.*

Implémentation.

- *Proposez une structure de données permettant d'enregistrer les objets, le prix et les valeurs de chacun d'eux.*
- *Implémentez une fonction accepte qui teste si l'on accepte de mettre un objet dans le sac.*
- *Proposez une fonction récursive qui débute par un test d'arrêt et l'enregistrement du sac si celui-ci a cout meilleur que celui qui était le meilleur jusqu'à là.*
- *Si l'on n'a pas terminé de remplir le sac, on doit alors parcourir les objets restant pour voir ce que l'on peut mettre. Dans le cas où un objet e peut être mis dans le sac il faut modifier nos paramètres appeler la fonction récursivement puis au retour effectuer le backtracking nécessaire.*

Exercice 4 : Reines

On dispose d'un échiquier 8x8 sur lequel on souhaite placer 8 reines qui ne soient pas en prise directe ; cela signifie qu'il ne doit pas y avoir deux reines sur une même ligne, ou sur une même colonne, ou sur une même parallèle à une diagonale. Chaque fois qu'une reine est placée sur l'échiquier on placera dans toutes les cases directement menacées par la reine, le numéro de la reine.

Dans notre exemple en plaçant la reine numéro 1 en position (0,0) la ligne 0, la colonne 0 et l'horizontale (i,i) sont menacées par cette reine. La reine 2 peut ensuite se placer dans toutes les cases qui ne contiennent pas la valeur 1. Si cette reine est placée en (3,2) elle menace un certain nombre de nouvelles cases.

Vous utiliserez un tableau à deux dimensions d'entiers correspondant aux cases de l'échiquier. Ce tableau est initialisé en mettant la valeur 0 dans toutes les cases, ce qui signifie que toutes les cases sont au départ disponible. Lorsqu'une case n'est plus libre, c'est-à-dire lorsqu'une reine déjà placée interdit de mettre une nouvelle

reine dans cette case, l'entier contenu par la case correspondante du tableau est le plus petit indice d'une reine interdisant cette case.

- Proposez deux structures de données une pour l'échiquier, et une pour indiquer les positions des reines ;
- Proposez un algorithme récursif qui permet d'obtenir une solution.

N	N	N	N	N	N	N	N
N	N	N	N	N	N	N	N
N	N	N	N	N	N	N	N
N	N	N	N	N	N	N	N
N	N	N	N	N	N	N	N
N	N	N	N	N	N	N	N
N	N	N	N	N	N	N	N
N	N	N	N	N	N	N	N

N	N	N	N	N	N	N	N
---	---	---	---	---	---	---	---

Fig.1

0	0	0	0	0	0	0	0
0	0	N	N	N	N	N	N
0	N	0	N	N	N	N	N
0	N	N	0	N	N	N	N
0	N	N	N	0	N	N	N
0	N	N	N	N	0	N	N
0	N	N	N	N	N	0	N
0	N	N	N	N	N	N	0

0	N	N	N	N	N	N	N
---	---	---	---	---	---	---	---

Fig.2

0	0	0	0	0	0	0	0
0	0	1	N	N	N	N	N
0	1	0	1	1	1	1	1
0	1	1	0	N	N	N	N
0	1	N	1	0	N	N	N
0	1	N	N	1	0	N	N
0	1	N	N	N	1	0	N
0	1	N	N	N	N	1	0

0	2	N	N	N	N	N	N
---	---	---	---	---	---	---	---

Fig.3

0	0	0	0	0	0	0	0
0	0	1	N	N	2	N	N
0	1	0	1	1	1	1	1
0	1	1	0	N	N	N	N
0	1	2	1	0	2	2	2
0	1	2	2	1	0	N	N
0	1	2	N	2	1	0	N
0	1	2	N	N	2	1	0

0	2	4	N	N	N	N	N
---	---	---	---	---	---	---	---

Fig.4

0	0	0	0	0	0	0	0
0	0	1	3	3	2	3	3
0	1	0	1	1	1	1	1
0	1	1	0	N	3	N	N
0	1	2	1	0	2	2	2
0	1	2	2	1	0	N	3
0	1	2	3	2	1	0	N
0	1	2	3	N	2	1	0

0	2	4	1	N	N	N	N
---	---	---	---	---	---	---	---

Fig.5

0	0	0	0	0	0	0	0
0	0	1	3	3	2	3	3
0	1	0	1	1	1	1	1
0	1	1	0	4	3	4	4
0	1	2	1	0	2	2	2
0	1	2	2	1	0	4	3
0	1	2	3	2	1	0	4
0	1	2	3	4	2	1	0

0	2	4	1	3	N	N	N
---	---	---	---	---	---	---	---

Fig.6

0	0	0	0	0	0	0	0
0	0	1	3	3	2	3	3
0	1	0	1	1	1	1	1
0	1	1	0	N	3	N	N
0	1	2	1	0	2	2	2
0	1	2	2	1	0	N	3
0	1	2	3	2	1	0	N
0	1	2	3	N	2	1	0

0	2	4	1	N	N	N	N
---	---	---	---	---	---	---	---

Fig.7

0	0	0	0	0	0	0	0
0	0	1	3	3	2	3	3
0	1	0	1	1	1	1	1
0	1	1	0	4	3	N	N
0	1	2	1	0	2	2	2
0	1	2	2	1	0	4	3
0	1	2	3	2	1	0	N
0	1	2	3	4	2	1	0

0	2	4	1	7	N	N	N
---	---	---	---	---	---	---	---

Fig.8